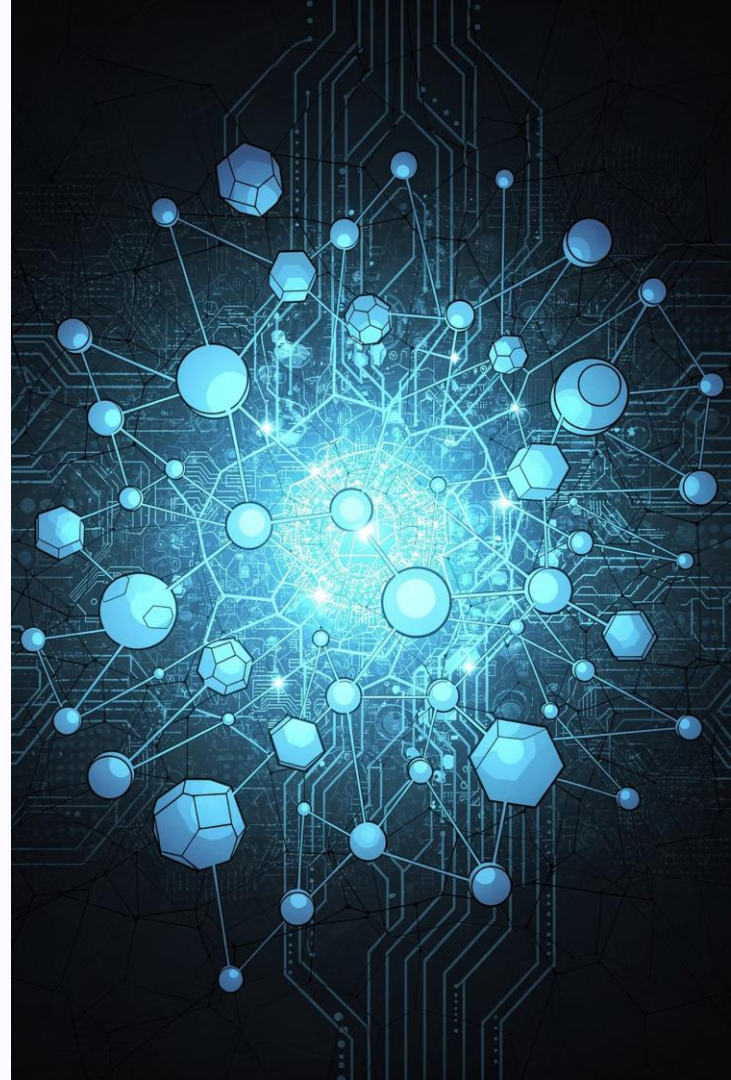


Algoritmos: La Ingeniería de la Eficiencia

Introducción al curso **Computación II** — Aprenderemos a medir, comparar y optimizar la lógica detrás de cada programa que existe.
¿Cómo sabemos cuándo un algoritmo es realmente bueno?

INTRODUCCIÓN A COMPUTACIÓN 2





Parte 1: El Arte del Algoritmo

Definición

Secuencia **finita y ordenada** de pasos para resolver un problema computacional.

Componentes

Entrada → Proceso → **Salida**.
Todo algoritmo tiene un inicio y un fin definido.

Objetivo

Resolver problemas de forma **eficiente**, utilizando la menor cantidad de recursos posibles.



CONCEPTOS CLAVE

Algoritmo vs Programa

Algoritmo

La **estrategia lógica** expresada en pseudocódigo. Independiente del lenguaje o plataforma.

Programa

La **implementación concreta** en un lenguaje específico: C++, Python, Java, etc.

❗ La lógica correcta del algoritmo es la base de todo rendimiento. Un mal algoritmo, bien codificado, sigue siendo ineficiente.

Capítulo 1: Análisis y Diseño de Algoritmos



Análisis

Medir tiempo y espacio

Diseño

Estrategias: dividir, iterar, recursar

Eficiencia

Métrica de calidad y escalabilidad

El análisis nos dice **cuánto cuesta** ejecutar un algoritmo; el diseño nos enseña **cómo construirlo mejor**. La eficiencia es la métrica que une ambas disciplinas.

¿Qué es la Eficiencia?



Tiempo de CPU

Número de operaciones ejecutadas en función del tamaño de la entrada n .



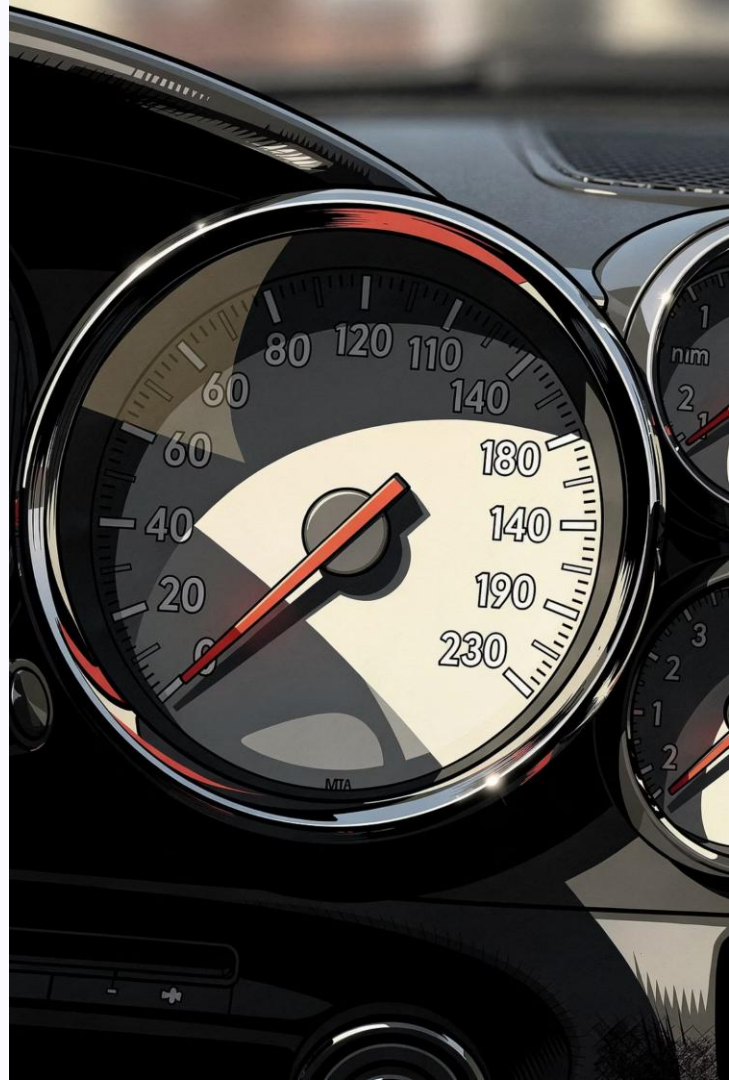
Memoria

Espacio en RAM requerido durante la ejecución. También se analiza en función de n .



Invarianza

Ignoramos el hardware específico. Un algoritmo eficiente lo es **en cualquier máquina**.



El Modelo de Computación RAM

El modelo **Random Access Machine (RAM)** es el marco teórico que usamos para analizar algoritmos de forma independiente al hardware real.

- Operaciones elementales: asignación, comparación, aritmética básica.
- Cada operación simple tiene **costo unitario igual a 1**.
- Acceso a memoria instantáneo — memoria ilimitada disponible.

📌 Este modelo nos permite analizar algoritmos matemáticamente sin depender de una CPU específica.

Operaciones Elementales

01

Asignación

$x = 5$

03

Aritmética

$a + b, a * b$

02

Comparación

$\text{if } (a > b)$

04

Acceso

$\text{arr}[i]$

Parte 2: Tiempo de Ejecución $T(n)$

¿Qué es n ?

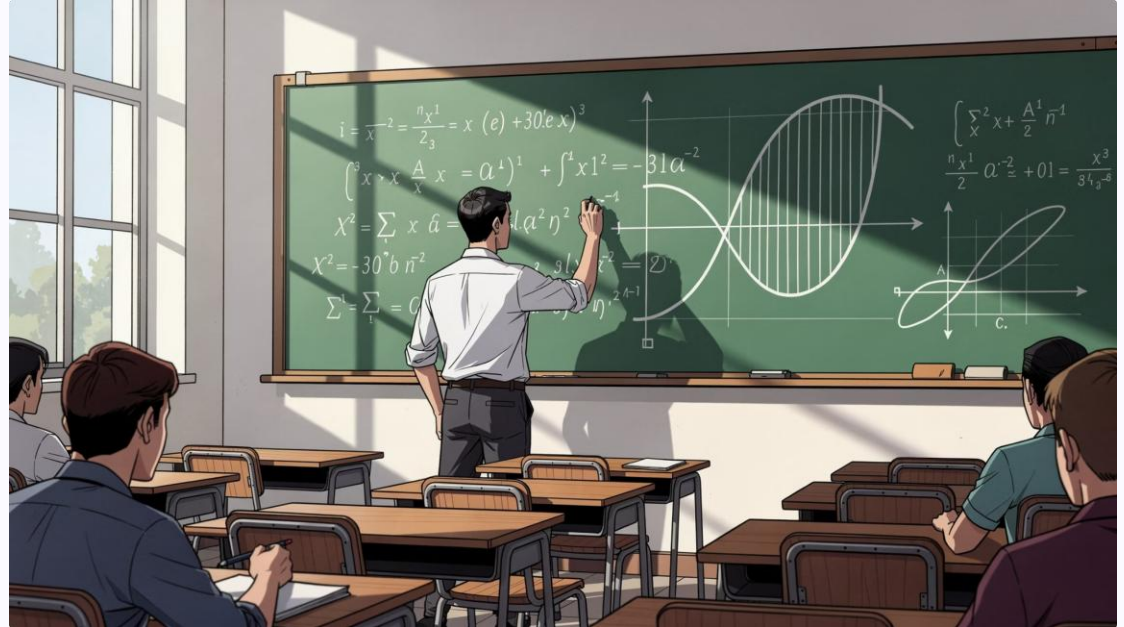
El **tamaño de la entrada**: cantidad de elementos a procesar, longitud del arreglo, número de nodos, etc.

¿Por qué no segundos?

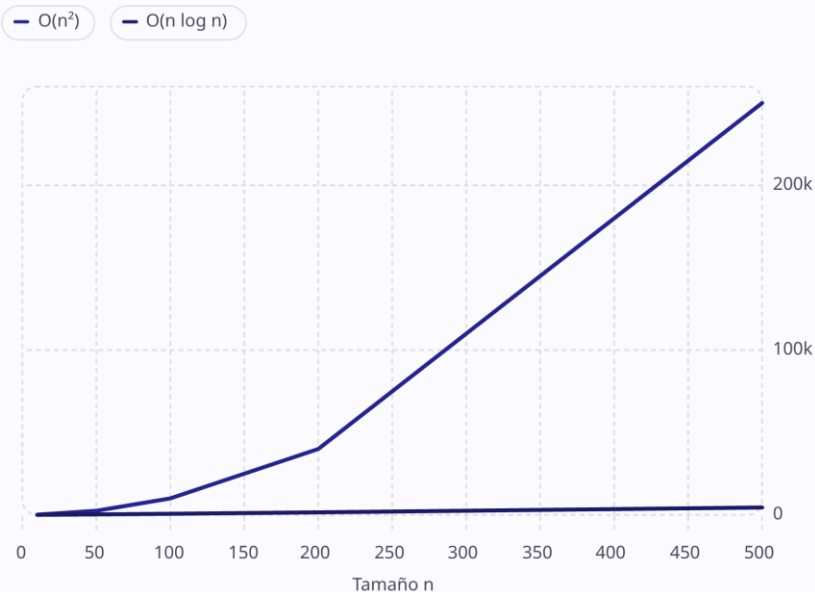
El tiempo en segundos varía según CPU.
 $T(n)$ mide **instrucciones ejecutadas**, una medida universal y precisa.

La función $T(n)$

Representación **matemática** de la cantidad de trabajo que realiza un algoritmo según el tamaño de su entrada.



El Comportamiento Asintótico



La Regla de Oro

A medida que **n crece**, las constantes y términos menores pierden relevancia. Solo importa la **tasa de crecimiento dominante**.

⚠ Un algoritmo $O(n^2)$ con $n=500$ ejecuta **55 veces más operaciones** que uno $O(n \log n)$. La brecha crece con n .

El análisis asintótico evalúa el comportamiento cuando $n \rightarrow \infty$.



Capítulo 2: El Análisis de Casos

● Mejor Caso

El escenario **ideal**. Ej: buscar el primer elemento de una lista. Proporciona el límite inferior del tiempo.

● Peor Caso

El escenario **más costoso** posible. Garantiza que el algoritmo nunca tardará más que $T_{\max}(n)$.

● Caso Promedio

El valor **esperado estadísticamente** bajo condiciones normales de uso. Requiere análisis probabilístico.



¿POR QUÉ EL PEOR CASO?

La Garantía de Seguridad

En sistemas críticos, el **peor caso es la única métrica que importa**. No podemos arriesgarnos a que un sistema falle bajo carga máxima.

→ **Sistemas de alarma nuclear:**
deben responder en tiempo
garantizado siempre.

→ **Sismógrafos en tiempo real:**
latencia máxima predecible es
obligatoria.

→ **Software bancario:**
transacciones deben
completarse dentro de un límite
conocido.

Parte 3: La Notación O Grande (Big-O)

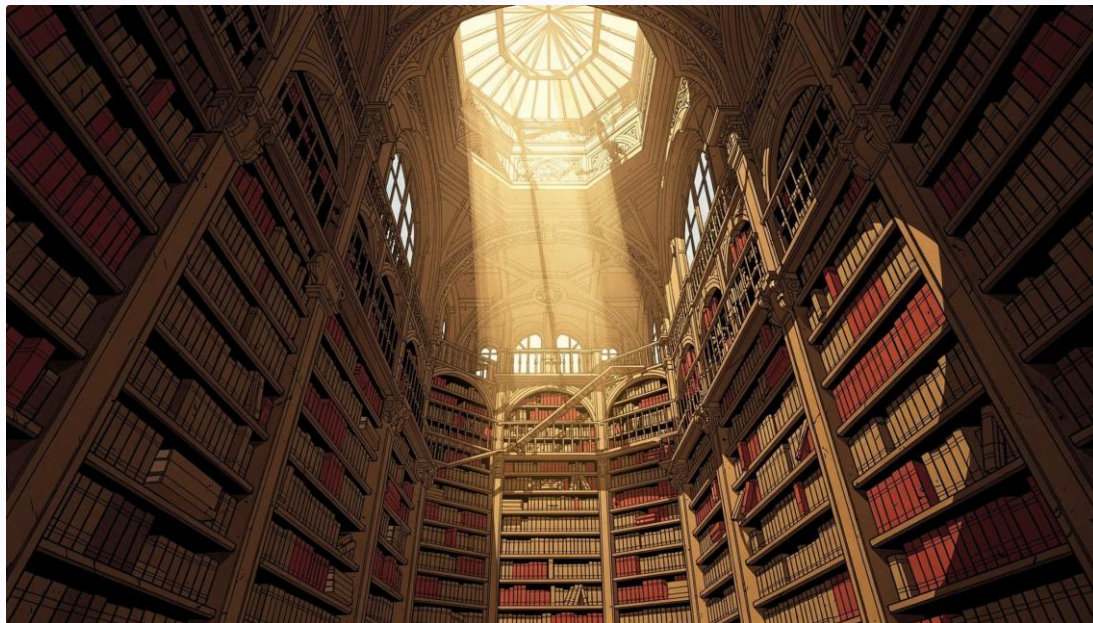
Definición Formal

Decimos que $f(n) \in O(g(n))$ si existen constantes positivas c y n_0 tales que:

$$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

Esto significa que $g(n)$ es una **cota superior** del comportamiento de $f(n)$ para entradas suficientemente grandes.

 Big-O describe el **peor escenario posible**: un límite que el algoritmo nunca superará.



Las Familias de Notación Asintótica



Analogía de Crecimiento



$O(1)$ — Constante

Sin importar cuántos datos haya, la operación tarda **siempre lo mismo**. Ej: acceder a `arr[i]`.



$O(\log n)$ — Logarítmico

Con cada paso se **elimina la mitad** del problema. Ej: búsqueda binaria en arreglo ordenado.



$O(n)$ — Lineal

Se recorre **cada elemento una vez**. El tiempo crece proporcionalmente con n . Ej: búsqueda secuencial.

El Muro de la Complejidad



$O(n^2)$ — Cuadrático

Doble bucle anidado. Duplicar n **cuadruplica** el tiempo. Ej: Bubble Sort, comparación de pares.



$O(2^n)$ — Exponencial

Cada elemento nuevo **duplica** el trabajo. Con $n=50$, se requieren más operaciones que átomos en el universo.

⊗ Un algoritmo $O(2^n)$ con $n=100$ ejecutaría $2^{100} \approx 10^{30}$ operaciones. Ningún servidor del planeta podría completarlo.



Capítulo 3: Ejemplos de Campo



La búsqueda binaria en una lista de **un millón de elementos** requiere como máximo **20 comparaciones**. La secuencial podría requerir un millón.

Insertion Sort — Análisis de Bucles

Mejor caso

Arreglo ya ordenado → **O(n)**. El bucle interno nunca se ejecuta.

Peor caso

Arreglo invertido → **O(n²)**. Cada elemento se compara con todos los anteriores.

Elección correcta

Conocer el contexto de uso determina **cuál algoritmo aplicar** en cada situación.

Herramientas de Medición

Método Experimental

Ejecutar el algoritmo y **medir el tiempo real** con un cronómetro de sistema.

Ventaja

Resultados tangibles y directos.

Desventaja

Varía según CPU, RAM, SO y carga del sistema.

Método Analítico

Contar operaciones elementales y derivar **$T(n)$ matemáticamente.**

Ventaja

Resultado universal, independiente del hardware.

Ventaja

Permite comparar algoritmos de forma objetiva y rigurosa.

✅ El método analítico es superior en ingeniería de software porque produce resultados **reproducibles, comparables y rigurosos.**



El Impacto en el Mundo Real



Bases de Datos

Los **índices B-Tree** reducen búsquedas de $O(n)$ a $O(\log n)$, transformando sistemas lentos en instantáneos.



Redes y Routing

Algoritmos como **Dijkstra** $O((V+E) \log V)$ permiten calcular rutas óptimas en internet en milisegundos.



Inteligencia Artificial

La eficiencia algorítmica determina si un modelo de **Machine Learning** entrena en horas o en meses.

La diferencia entre un buen ingeniero y un gran ingeniero no es solo que el código funcione — es que **funcione a escala**.